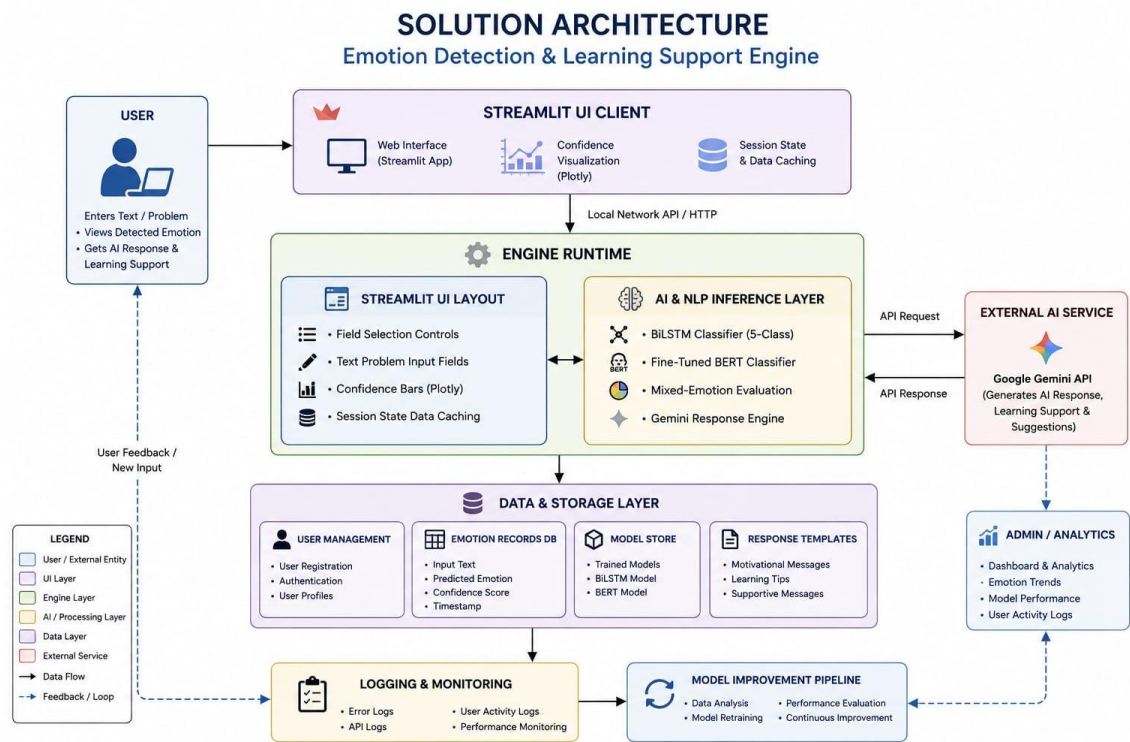


Project Requirement Analysis Phase Technology Stack (Architecture & Stack)

Date	01 July 2026
Team ID	SWTID-2026-2152
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	4 Marks

1. Architectural Design & System Flow

The platform is engineered using a modular, decoupled AI-to-UI architecture optimized for low-latency inference and real-time session tracking.



The application separates text preprocessing and model execution pipelines from the user presentation layer, ensuring that memory-heavy transformer models load into a persistent local cache without degrading interface responsiveness.

2. Comprehensive Technology Stack

A. Frontend & Interface Layer

- **Framework: Streamlit**
 - **Implementation Details:** Leveraged for rapid, highly responsive layout assembly. Uses structural session state management (`st.session_state`) to maintain user interaction data, input fields, and processing results across runtime reruns.

- **Data Visualization Engine: Plotly Charts**
 - **Implementation Details:** Handles generation of dynamic, interactive confidence bars and analytics tabs, displaying emotion probability distributions (BiLSTM vs. BERT outputs) using localized data caching.
- **Interface Elements:** Custom native form controls, loading spinners, run/clear triggers, and custom field selectors built into a unified prediction schema.

B. Machine Learning & Emotion Detection Pipeline (Dual-Model Architecture)

- **Sequential Classifier:** Bidirectional Long Short-Term Memory (BiLSTM)
 - **Implementation Details:** Trained using Kaggle GPU environments and exported locally. Runs a 5-Class Softmax output layer to categorize input text into structural baseline emotional frequencies.
- **Contextual Transformer:** Fine-Tuned BERT Classifier
 - **Implementation Details:** Utilizes domain-adaptive fine-tuning and specialized class weighting to counteract dataset imbalances, uncovering deep contextual sentiment and keyword nuances from text reflections.
- **Hybrid Evaluation Layer:** Mixed-Emotion Detection Engine
 - **Implementation Details:** Evaluates outputs from both models simultaneously. If a secondary emotion achieves a threshold score of $\geq 15\%$, it is captured under a unified prediction schema to identify layered student mindsets (e.g., confused yet engaged).

C. Generative AI Guidance & Regeneration Engine

- **Core Orchestrator:** Google Gemini API
 - **Implementation Details:** Integrates securely using an encapsulated API key managed through specialized local configurations (.env).
- **Prompt Architecture:** Field-Aware Prompt Generation
 - **Implementation Details:** Dynamically constructs prompts combining the student's *Academic Field*, specific *Problem Input*, *Detected Emotion*, and *Confidence Score* to feed into Gemini.
- **Response Fallback System:** Employs programmatic template fallbacks to guarantee empathetic response generation even during rate limits or network interruptions.

D. Data Persistence & Performance Management

- **Data Serialization & Formats:** CSV Persistence Engine
 - **Implementation Details:** Records session histories, text logs, confidence scores, and interaction timelines into structural CSV files for continuous offline learning and metric analytics.
- **Optimization Layer:** Cached Model Loading
 - **Implementation Details:** Uses memory caching flags to keep heavy model weights pre-loaded in environment memory, eliminating redundant disk-read operations and maintaining rapid inference.

E. Environment & Development Pipeline

- **Core Runtime:** Python (Virtual Environments)
 - **Implementation Details:** Isolated environment topologies utilizing localized .env resource configuration files to manage model directories, security credentials, and system dependencies.
- **Training & Prototyping Platform:** Kaggle Ecosystem
 - **Implementation Details:** Utilized remote GPU acceleration for initial tokenizer building, vocabulary alignment, model training, and performance validation before local export.